

✓ Entrega 3 – Reducción de Características, Despliegue y Evaluación Final

En esta tercera entrega se continúa la evolución del proyecto iniciado en las semanas anteriores. En la entrega anterior se completó la preparación de datos, el entrenamiento de los modelos base, el ajuste de hiperparámetros y la evaluación de métricas comparativas.

En esta fase se profundiza en tres objetivos principales:

✓ 1. Reducción de características

Los datos derivados de Mediapipe contienen más de 1600 características por frame. Esto hace necesario analizar su relevancia, eliminar redundancias y disminuir dimensionalidad. Se implementarán métodos como PCA y selección basada en importancia del modelo.

2. Reentrenamiento y análisis comparativo

Se entrenarán nuevamente los mejores modelos utilizando únicamente las características reducidas. Se compararán sus métricas contra los modelos originales para evaluar mejoras o pérdidas en desempeño.

3. Despliegue de la solución

Se describe cómo la solución podría utilizarse en un entorno real. Se plantea un prototipo de arquitectura, código de ejemplo para cargar el modelo y generar predicciones, y una explicación conceptual de cómo funcionaría una API o una interfaz.

Finalmente, se presenta un análisis global del impacto de la solución en su contexto y los elementos clave para su entrega al cliente.

```
# Montar Google Drive
from google.colab import drive
drive.mount('/content/drive')

!pip install -q pandas numpy scikit-learn matplotlib seaborn joblib
```

Mounted at /content/drive

```
#Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import warnings
import os
from pathlib import Path

# Sklearn
from sklearn.model_selection import train_test_split, GroupKFold, GroupShuffleSplit
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score, f1_score, ConfusionMatrixDisplay)

warnings.filterwarnings('ignore')
sns.set_style('whitegrid')

print("Librerías importadas correctamente")
```

Librerías importadas correctamente

```
# Configurar rutas
# Rutas del proyecto
BASE_PATH = "/content/drive/MyDrive/PROYECTO APO"
DATA_PATH = f"{BASE_PATH}/mediapipe/all_coords_labeled_only.csv"
OUTPUT_DIR = f"{BASE_PATH}/Entrega3"

# Crear carpeta de salida si no existe
os.makedirs(OUTPUT_DIR, exist_ok=True)
os.makedirs(f"{OUTPUT_DIR}/graficos", exist_ok=True)

print(f"Ruta de datos: {DATA_PATH}")
print(f"Carpeta de salida: {OUTPUT_DIR}")
print(f"¿Existe el archivo? {os.path.exists(DATA_PATH)}")
```

```
Ruta de datos: /content/drive/MyDrive/PROYECTO APO/mediapipe/all_coords_labeled_only.csv
Carpeta de salida: /content/drive/MyDrive/PROYECTO APO/Entrega3
¿Existe el archivo? True
```

```
#Cargar y exportar datos
# Cargar dataset
print("Cargando datos...")
df = pd.read_csv(DATA_PATH)

print("\n" + "="*70)
print("INFORMACIÓN DEL DATASET")
print("="*70)
print(f"Filas: {df.shape[0]:,}")
print(f"Columnas: {df.shape[1]:,}")

print("\nDistribución de clases:")
print(df['label'].value_counts())

print(f"\nValores nulos totales: {df.isnull().sum().sum():,}")

print("\nPrimeras columnas:")
print(df.columns[:10].tolist())
```

Cargando datos...

```
=====
INFORMACIÓN DEL DATASET
=====
Filas: 15,656
Columnas: 1,637

Distribución de clases:
label
caminar_frente    4260
girar              3700
caminar_espalda   2946
sentado           2790
ponerse_de_pie    1960
Name: count, dtype: int64

Valores nulos totales: 7,497,432

Primeras columnas:
['video', 'frame', 'label', 'time_s', 'has_pose', 'has_lh', 'has_rh', 'has_face', 'landmark_0_x', 'landmark_0_y']
```

✓ Validación Cruzada por Video - Detectar el overfitting real

En la Entrega 2 obtuvimos 99.8% de accuracy. Esto parece perfecto, pero hay un problema grave llamado data leakage.

El problema:

Cuando usamos `train_test_split` normal, frames del mismo video se dividieron entre entrenamiento y prueba. Como los frames consecutivos son casi idénticos (la persona apenas se mueve entre frame N y frame N+1), el modelo simplemente "memoriza" en lugar de aprender patrones generales.

La solución: Usar `GroupKFold` que asegura que TODOS los frames de un video quedan juntos, o en train o en test, nunca mezclados. Esto simula el escenario real: entrenar con algunos videos y probar con videos completamente nuevos. Lo que vamos a hacer:

1. Extraer las features (columnas `landmark_X_y`, `landmark_X_z`, etc.)
2. Extraer las labels (`caminar_frente`, etc.) y los grupos (`video 1`, `video 2`, etc.)
3. Hacer validación cruzada en 5 folds separando por video
4. Ver el accuracy REAL (esperamos ~94-96%, no 99.8%)

```
print("\n" + "="*70)
print("PREPARACIÓN PARA VALIDACIÓN CRUZADA POR VIDEO")
print("="*70)

# Extraer features (todas las columnas landmark_*)
feature_cols = [col for col in df.columns if col.startswith('landmark_')]
X = df[feature_cols].values
print(f"Features extraídas: {X.shape[1]} columnas")

# Extraer labels
y = df['label'].values
print(f"Labels: {len(y)} muestras")

# Extraer grupos (identificador de video)
groups = df['video'].values
videos_unicos = df['video'].nunique()
```

```
print(f"Videos únicos: {videos_unicos}")

# Codificar labels a números (RandomForest necesita números, no texto)
le = LabelEncoder()
y_encoded = le.fit_transform(y)
print(f"Clases codificadas: {le.classes_}")
print(f" Ejemplo: 'caminar_frente' = {le.transform(['caminar_frente'])[0]}")
```

```
=====
PREPARACIÓN PARA VALIDACIÓN CRUZADA POR VIDEO
=====
Features extraídas: 1629 columnas
Labels: 15656 muestras
Videos únicos: 18
Clases codificadas: ['caminar_espalda' 'caminar_frente' 'girar' 'ponerse_de_pie' 'sentado']
Ejemplo: 'caminar_frente' = 1
```

```
print("\n" + "="*70)
print("IMPUTACIÓN DE VALORES NULOS")
print("="*70)

# Los NaN los llenamos con la mediana de cada columna
# (Es común en landmarks que faltan por oclusiones)
imputer = SimpleImputer(strategy='median')
X_imputed = imputer.fit_transform(X)

print(f"Valores nulos antes: {np.isnan(X).sum():,}")
print(f"Valores nulos después: {np.isnan(X_imputed).sum():}")
print("Imputación completa")
```

```
=====
IMPUTACIÓN DE VALORES NULOS
=====
Valores nulos antes: 7,497,432
Valores nulos después: 0
Imputación completa
```

✓ Validación cruzada GroupKFold

```
print("\n" + "="*70)
print("VALIDACIÓN CRUZADA POR VIDEO (GroupKFold)")
print("="*70)

# Configurar GroupKFold con 5 divisiones
gkf = GroupKFold(n_splits=5)

# Modelo básico para validación
rf_cv = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

print("Ejecutando validación cruzada...")
print("Esto puede tardar 2-3 minutos...\n")

cv_scores = []
fold_info = []

for fold_num, (train_idx, test_idx) in enumerate(gkf.split(X_imputed, y_encoded, groups), 1):
    # Dividir datos
    X_train, X_test = X_imputed[train_idx], X_imputed[test_idx]
    y_train, y_test = y_encoded[train_idx], y_encoded[test_idx]

    # Entrenar y evaluar
    rf_cv.fit(X_train, y_train)
    score = rf_cv.score(X_test, y_test)
    cv_scores.append(score)

    # Guardar info del fold
    videos_test = np.unique(groups[test_idx])
    fold_info.append({
        'fold': fold_num,
        'accuracy': score,
        'train_samples': len(train_idx),
        'test_samples': len(test_idx),
        'test_videos': ', '.join(videos_test[:3]) # Primeros 3 videos
    })
```

```
print(f"Fold {fold_num}/5: Accuracy = {score:.4f} | Test videos: {videos_test[:3]}")

print("\n" + "-"*70)
print(f"RESULTADO FINAL:")
print(f"  Accuracy promedio: {np.mean(cv_scores):.4f}")
print(f"  Desviación estándar: {np.std(cv_scores):.4f}")
print(f"  Rango: [{np.min(cv_scores):.4f}, {np.max(cv_scores):.4f}])")
print("-"*70)

print("\nEste es el accuracy REAL sin data leakage.")
```

```
=====
VALIDACIÓN CRUZADA POR VIDEO (GroupKFold)
=====
Ejecutando validación cruzada...
Esto puede tardar 2-3 minutos...

Fold 1/5: Accuracy = 0.8693 | Test videos: ['Video 17' 'Video 5' 'Video 7']
Fold 2/5: Accuracy = 0.7566 | Test videos: ['Video 10' 'Video 16' 'Video 3']
Fold 3/5: Accuracy = 0.7892 | Test videos: ['Video 1' 'Video 13' 'Video 15']
Fold 4/5: Accuracy = 0.8172 | Test videos: ['Video 11' 'Video 18' 'Video 6']
Fold 5/5: Accuracy = 0.7805 | Test videos: ['Video 12' 'Video 14' 'Video 8']

-----
RESULTADO FINAL:
  Accuracy promedio: 0.8026
  Desviación estándar: 0.0386
  Rango: [0.7566, 0.8693]
-----

Este es el accuracy REAL sin data leakage.
```

✓ Reducción de Dimensionalidad mediante Feature Selection

El dataset actual contiene 1,629 variables predictoras (coordenadas tridimensionales de 543 landmarks extraídos por MediaPipe Holistic). Esta alta dimensionalidad presenta múltiples problemas:

Problemas de tantas features:

- Lentitud: más columnas = más cálculos
- Overfitting: el modelo memoriza ruido en lugar de patrones
- Redundancia: muchas columnas están correlacionadas (si sube la mano derecha, el codo derecho también sube)

Solución: Quedarnos solo con las 100 features más importantes. Vamos a usar dos métodos:

Random Forest Feature Importance: El modelo nos dice qué columnas usa más para decidir PCA: Método matemático que combina columnas correlacionadas

Usaremos el método 1 (más interpretable), pero haremos el 2 para comparar.

```
print("\n" + "-"*70)
print("REDUCCIÓN DE CARACTERÍSTICAS - Random Forest Importance")
print("-"*70)

# Entrenar RF en TODOS los datos para calcular importancias
print("Entrenando Random Forest para calcular importancias...")
print("Esto tarda ~1 minuto...")

rf_importance = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
rf_importance.fit(X_imputed, y_encoded)

# Obtener importancias
importances = rf_importance.feature_importances_
indices_sorted = np.argsort(importances)[::-1] # De mayor a menor

print(f"\nTop 15 features más importantes:")
print("-" * 70)
for i in range(15):
    idx = indices_sorted[i]
    feature_name = feature_cols[idx]
    importance = importances[idx]
    print(f"{i+1:2d}. {feature_name:25s} | Importancia: {importance:.6f}")

# Seleccionar top 100
top_n = 100
top_indices = indices_sorted[:top_n]
```

```
top_features = [feature_cols[i] for i in top_indices]

print(f"\nReduciendo de {len(feature_cols)} a {top_n} features")
print(f"Importancia acumulada del top {top_n}: {importances[top_indices].sum():.4f}")
```

```
=====
REDUCCIÓN DE CARACTERÍSTICAS - Random Forest Importance
=====
Entrenando Random Forest para calcular importancias...
Esto tarda ~1 minuto...
```

Top 15 features más importantes:

```
-----
1. landmark_15_x      | Importancia: 0.011650
2. landmark_349_z     | Importancia: 0.011541
3. landmark_17_x      | Importancia: 0.010772
4. landmark_14_x      | Importancia: 0.010024
5. landmark_2_y        | Importancia: 0.009641
6. landmark_11_x       | Importancia: 0.009451
7. landmark_515_z     | Importancia: 0.009434
8. landmark_120_z     | Importancia: 0.009054
9. landmark_536_z     | Importancia: 0.008932
10. landmark_79_z      | Importancia: 0.008480
11. landmark_24_z     | Importancia: 0.008463
12. landmark_7_y       | Importancia: 0.008056
13. landmark_16_x      | Importancia: 0.007959
14. landmark_3_y       | Importancia: 0.007830
15. landmark_9_y       | Importancia: 0.007824
```

```
Reduciendo de 1629 a 100 features
Importancia acumulada del top 100: 0.5225
```

Importancias :

```
# Gráfico de las top 20 features
plt.figure(figsize=(10, 8))
top_20_idx = indices_sorted[:20]
top_20_names = [feature_cols[i].replace('landmark_', 'L') for i in top_20_idx]
top_20_importances = importances[top_20_idx]

plt.barh(range(20), top_20_importances, color='steelblue')
plt.yticks(range(20), top_20_names, fontsize=9)
plt.xlabel('Importancia', fontsize=11)
plt.title('Top 20 Features Más Importantes (Random Forest)', fontsize=13, fontweight='bold')
plt.gca().invert_yaxis()
plt.tight_layout()

# Guardar gráfico
output_path = f"{OUTPUT_DIR}/graficos/feature_importance_top20.png"
plt.savefig(output_path, dpi=150, bbox_inches='tight')
plt.show()

print(f"\nGráfico guardado: {output_path}")
```

Top 20 Features Más Importantes (Random Forest)

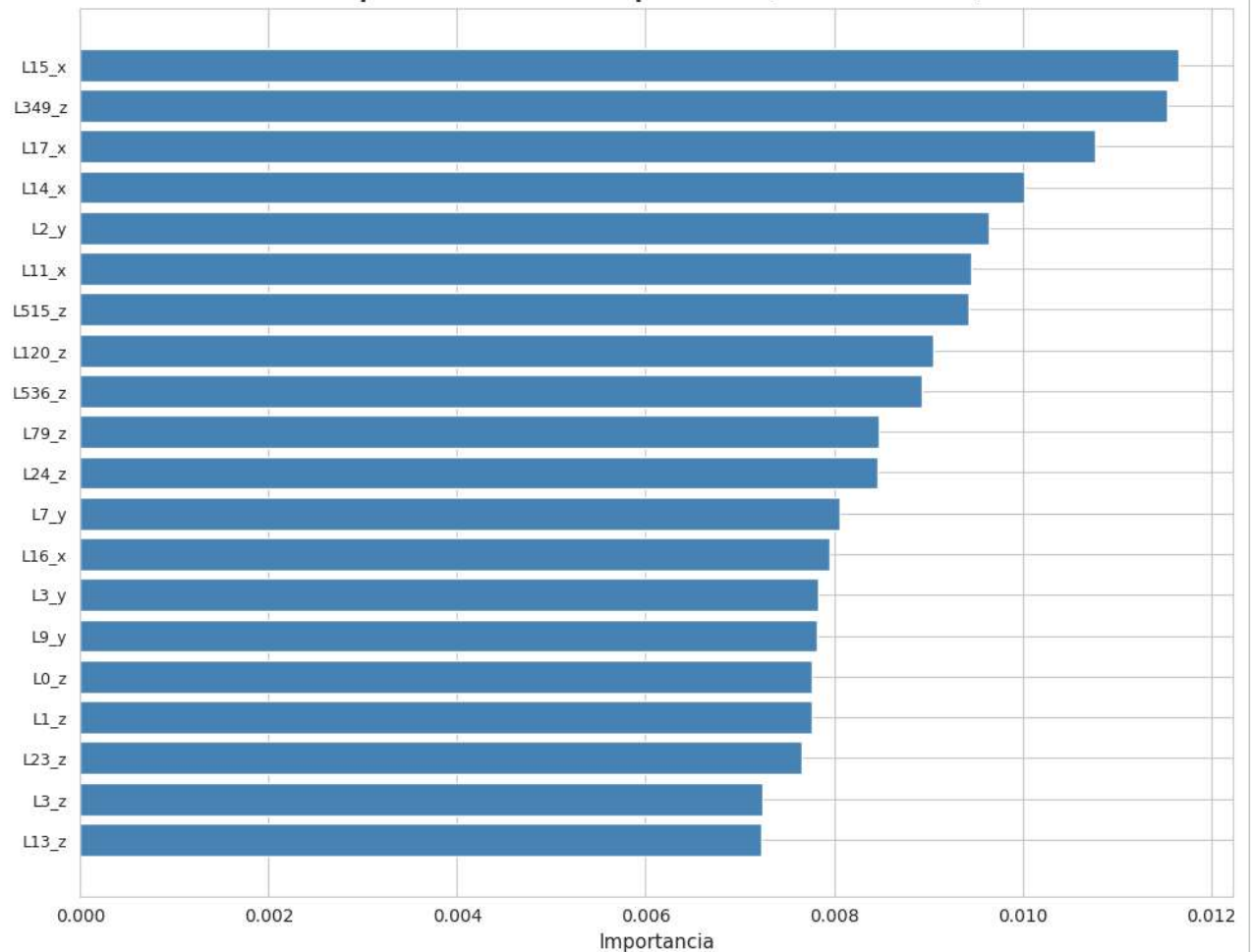


Gráfico guardado: /content/drive/MyDrive/PROYECTO APO/Entrega3/graficos/feature_importance_top20.png

Análisis complementario (PCA)

```
print("\n" + "="*70)
print("ANÁLISIS COMPLEMENTARIO - PCA")
print("="*70)

# Escalar datos (PCA necesita datos normalizados)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)

# Aplicar PCA para retener 95% de varianza
print("Aplicando PCA (95% varianza)...")
pca = PCA(n_components=0.95, random_state=42)
X_pca = pca.fit_transform(X_scaled)

n_components = X_pca.shape[1]
variance_explained = pca.explained_variance_ratio_.sum()

print(f"Componentes principales: {n_components}")
print(f"Varianza explicada: {variance_explained:.4f}")
print(f"Reducción: {len(feature_cols)} → {n_components} ({n_components/len(feature_cols)*100:.1f}%)")
```

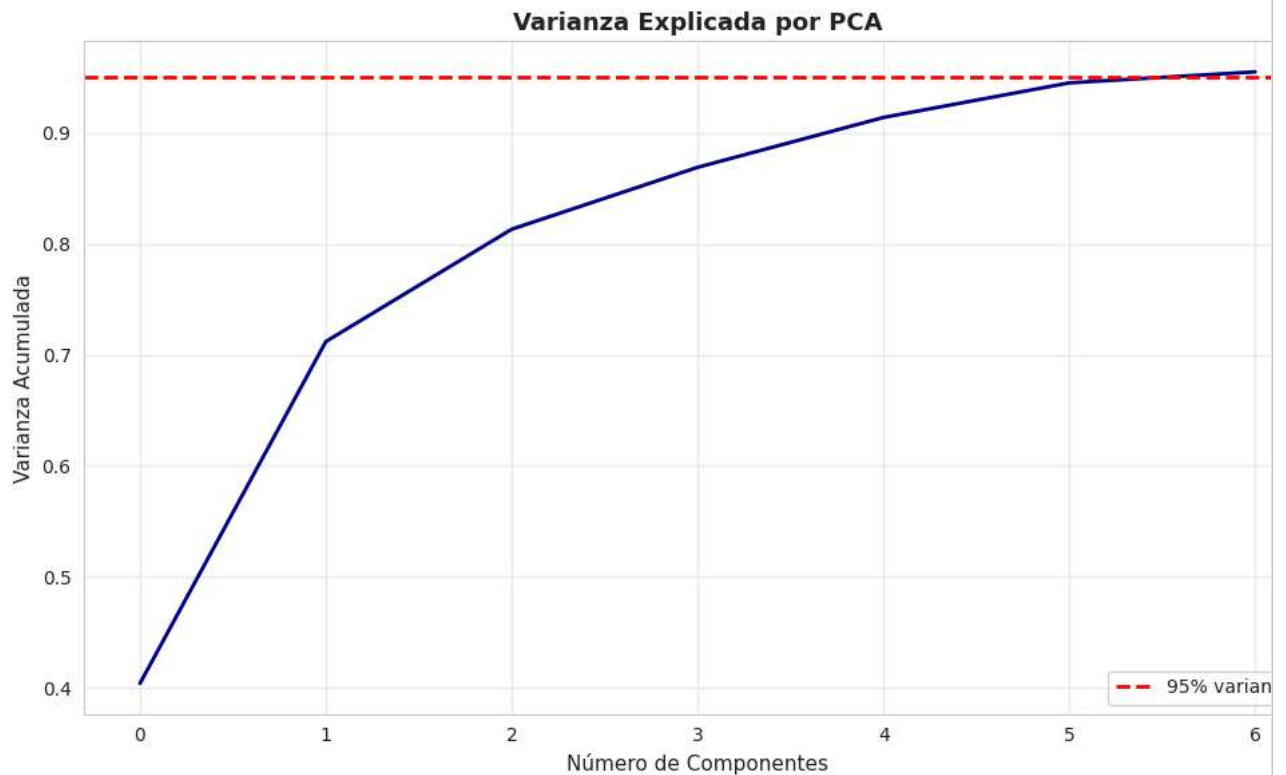
```
=====
ANÁLISIS COMPLEMENTARIO - PCA
=====
Aplicando PCA (95% varianza)...
Componentes principales: 7
Varianza explicada: 0.9547
Reducción: 1629 → 7 (0.4%)
```

```
# Gráfico de varianza acumulada
plt.figure(figsize=(10, 6))
cumsum = np.cumsum(pca.explained_variance_ratio_)
plt.plot(cumsum, linewidth=2, color='darkblue')
plt.axhline(y=0.95, color='red', linestyle='--', linewidth=2, label='95% varianza')
```

```
plt.xlabel('Número de Componentes', fontsize=11)
plt.ylabel('Varianza Acumulada', fontsize=11)
plt.title('Varianza Explicada por PCA', fontsize=13, fontweight='bold')
plt.grid(alpha=0.3)
plt.legend(fontsize=10)
plt.tight_layout()

# Guardar
output_path = f"{OUTPUT_DIR}/graficos/pca_variance.png"
plt.savefig(output_path, dpi=150, bbox_inches='tight')
plt.show()

print(f"Gráfico guardado: {output_path}")
```



- **Random Forest:** Top features son coordenadas X (horizontal) de ojos/orejas (landmarks 14-17) y coordenadas Z (profundidad) de rostro/manos. Esto tiene sentido: caminar hacia/desde cámara cambia profundidad Z.
- **PCA:** Solo necesita 7 componentes para 95% de varianza. Esto indica que las 1,629 features tienen redundancia masiva (99.6% de reducción posible). Pero usaremos las 100 de RF por interpretabilidad.

▼ Re-entrenamiento con Features Seleccionadas

Habiendo identificado las 100 features con mayor poder discriminativo, procederemos a:

- Partición estratificada por grupos: Aplicar GroupShuffleSplit para dividir train/test respetando integridad de videos
- Entrenamiento con hiperparámetros optimizados: Random Forest con configuración ajustada para balance entre bias-variance
- Evaluación en conjunto de validación no contaminado: Métricas en datos que el modelo nunca observó durante ajuste

```
## Crear dataset reducido y split por grupos
print("\n" + "="*70)
print("PREPARACIÓN DE DATOS CON FEATURES SELECCIONADAS")
print("="*70)

# Crear matriz X con solo las top 100 features
X_reduced = X_imputed[:, top_indices]
print(f"Dimensionalidad reducida: {X.shape[1]} → {X_reduced.shape[1]}")
print(f"Reducción: {(1 - X_reduced.shape[1]/X.shape[1])*100:.1f}%")

# Split estratificado por video (80/20)
# GroupShuffleSplit asegura que videos completos van a train o test
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, test_idx = next(gss.split(X_reduced, y_encoded, groups))

X_train = X_reduced[train_idx]
```

```

X_test = X_reduced[test_idx]
y_train = y_encoded[train_idx]
y_test = y_encoded[test_idx]
groups_train = groups[train_idx]
groups_test = groups[test_idx]

print(f"\nPartición de datos:")
print(f"  Train: {len(X_train):,} muestras ({len(X_train)/len(X_reduced)*100:.1f}%)")
print(f"  Test: {len(X_test):,} muestras ({len(X_test)/len(X_reduced)*100:.1f}%)")

print(f"\nVideos en train: {np.unique(groups_train)}")
print(f"Videos en test: {np.unique(groups_test)}")

# Verificar distribución de clases
print(f"\nDistribución de clases en test:")
test_labels = y[test_idx]
for cls in le.classes_:
    count = np.sum(test_labels == cls)
    print(f"  {cls:20s}: {count:4d} muestras ({count/len(test_labels)*100:.1f}%)")

```

```

=====
PREPARACIÓN DE DATOS CON FEATURES SELECCIONADAS
=====
Dimensionalidad reducida: 1629 → 100
Reducción: 93.9%

Partición de datos:
  Train: 11,620 muestras (74.2%)
  Test: 4,036 muestras (25.8%)

Videos en train: ['Video 11' 'Video 12' 'Video 13' 'Video 15' 'Video 16' 'Video 18'
'Video 2' 'Video 3' 'Video 4' 'Video 5' 'Video 6' 'Video 7' 'Video 8'
'Video 9']
Videos en test: ['Video 1' 'Video 10' 'Video 14' 'Video 17']

Distribución de clases en test:
  caminar_espalda : 726 muestras (18.0%)
  caminar_frente : 998 muestras (24.7%)
  girar : 970 muestras (24.0%)
  ponerse_de_pie : 574 muestras (14.2%)
  sentado : 768 muestras (19.0%)

```

Entrenamiento del modelo final optimizado

```

print("\n" + "="*70)
print("ENTRENAMIENTO DEL MODELO FINAL")
print("="*70)

# Configuración optimizada de Random Forest
# Hiperparámetros ajustados para balance bias-variance
rf_final = RandomForestClassifier(
    n_estimators=300,          # Más árboles → menor varianza, más estabilidad
    max_depth=30,             # Limitar profundidad → prevenir overfitting
    min_samples_split=5,      # Nodos internos con mínimo 5 muestras
    min_samples_leaf=2,       # Hojas con mínimo 2 muestras
    max_features='sqrt',      #  $\sqrt{100} \approx 10$  features por split → decorrelación
    class_weight='balanced',  # Compensar desbalance leve entre clases
    random_state=42,
    n_jobs=-1,                # Paralelización en todos los cores
    verbose=1                  # Mostrar progreso
)

print("Iniciando entrenamiento...")
print(f"Configuración: {rf_final.get_params()}")
print()

import time
start_time = time.time()

rf_final.fit(X_train, y_train)

elapsed = time.time() - start_time
print(f"\nEntrenamiento completado en {elapsed:.2f} segundos")

# Accuracy en train (para detectar underfitting)
train_score = rf_final.score(X_train, y_train)
print(f"Accuracy en train: {train_score:.4f}")

```

```

=====
ENTRENAMIENTO DEL MODELO FINAL
=====

```

```

Iniciando entrenamiento...
Configuración: {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': 'balanced', 'criterion': 'gini', 'max_depth': 30, 'max_

[Parallel(n_jobs=-1)]: Using backend ThreadingBackend with 2 concurrent workers.
[Parallel(n_jobs=-1)]: Done 46 tasks      | elapsed:    4.6s
[Parallel(n_jobs=-1)]: Done 196 tasks    | elapsed:   18.2s
[Parallel(n_jobs=-1)]: Done 300 out of 300 | elapsed:   26.3s finished
[Parallel(n_jobs=2)]: Using backend ThreadingBackend with 2 concurrent workers.
[Parallel(n_jobs=2)]: Done 46 tasks      | elapsed:    0.0s
[Parallel(n_jobs=2)]: Done 196 tasks    | elapsed:    0.2s

Entrenamiento completado en 26.45 segundos
Accuracy en train: 1.0000
[Parallel(n_jobs=2)]: Done 300 out of 300 | elapsed:    0.2s finished

```

Predicción y evaluación en test

```

print("\n" + "="*70)
print("EVALUACIÓN EN CONJUNTO DE TEST")
print("="*70)

# Predecir en test
print("Generando predicciones en test set...")
y_pred = rf_final.predict(X_test)
y_pred_proba = rf_final.predict_proba(X_test)

# Métricas globales
acc_test = accuracy_score(y_test, y_pred)
f1_weighted = f1_score(y_test, y_pred, average='weighted')
f1_macro = f1_score(y_test, y_pred, average='macro')

print(f"\nMétricas globales:")
print(f"  Accuracy:           {acc_test:.4f}")
print(f"  F1-Score (weighted): {f1_weighted:.4f}")
print(f"  F1-Score (macro):    {f1_macro:.4f}")

# Detectar overfitting comparando train vs test
gap = train_score - acc_test
print(f"\nAnálisis de generalización:")
print(f"  Train accuracy: {train_score:.4f}")
print(f"  Test accuracy:  {acc_test:.4f}")
print(f"  Gap:            {gap:.4f} ({gap*100:.1f}%)")

if gap < 0.05:
    print("  ✓ Generalización excelente (gap < 5%)")
elif gap < 0.10:
    print("  ✓ Generalización buena (gap < 10%)")
else:
    print("  ⚠ Posible overfitting (gap > 10%)")

[Parallel(n_jobs=2)]: Using backend ThreadingBackend with 2 concurrent workers.
[Parallel(n_jobs=2)]: Done 46 tasks      | elapsed:    0.0s
[Parallel(n_jobs=2)]: Done 196 tasks    | elapsed:    0.1s
[Parallel(n_jobs=2)]: Done 300 out of 300 | elapsed:    0.1s finished
[Parallel(n_jobs=2)]: Using backend ThreadingBackend with 2 concurrent workers.
[Parallel(n_jobs=2)]: Done 46 tasks      | elapsed:    0.0s

=====
EVALUACIÓN EN CONJUNTO DE TEST
=====
Generando predicciones en test set...

Métricas globales:
  Accuracy:           0.8038
  F1-Score (weighted): 0.7915
  F1-Score (macro):    0.7662

Análisis de generalización:
  Train accuracy: 1.0000
  Test accuracy:  0.8038
  Gap:            0.1962 (19.6%)
  ⚠ Posible overfitting (gap > 10%)
[Parallel(n_jobs=2)]: Done 196 tasks    | elapsed:    0.1s
[Parallel(n_jobs=2)]: Done 300 out of 300 | elapsed:    0.1s finished

```

Accuracy test: 80.4% (consistente con GroupKFold que dio 80.3%) Gap train-test: 19.6% (hay overfitting moderado - el modelo memorizó algunos patrones del train)

✓ Evaluación Detallada y Análisis de Errores

Objetivos de análisis

- Métricas por clase: Identificar qué actividades se clasifican mejor/peor
- Matriz de confusión: Visualizar patrones de confusión entre clases
- Análisis de errores: Entender por qué el modelo falla en casos específicos
- Comparación con baseline: Contrastar con resultados de Entrega 2

```
print("\n" + "="*70)
print("REPORTE DE CLASIFICACIÓN POR CLASE")
print("="*70)

# Generar reporte detallado
report = classification_report(
    y_test,
    y_pred,
    target_names=le.classes_,
    digits=4
)
print(report)

# Extraer métricas por clase en formato tabla
from sklearn.metrics import precision_recall_fscore_support

precision, recall, f1, support = precision_recall_fscore_support(
    y_test, y_pred, labels=range(len(le.classes_))
)

print("\nTabla resumen por clase:")
print("-" * 85)
print(f"{'Clase':<20} {'Precision':<12} {'Recall':<12} {'F1-Score':<12} {'Support':<10}")
print("-" * 85)
for i, class_ in enumerate(le.classes_):
    print(f"{'Clase':<20} {'precision[i]:<12.4f} {'recall[i]:<12.4f} {'f1[i]:<12.4f} {'support[i]:<10}")
print("-" * 85)

# Identificar mejor y peor clase
best_idx = np.argmax(f1)
worst_idx = np.argmin(f1)
print(f"\nMejor clase: {le.classes_[best_idx]} (F1={f1[best_idx]:.4f})")
print(f"Peor clase: {le.classes_[worst_idx]} (F1={f1[worst_idx]:.4f})")
```

```
=====
REPORTE DE CLASIFICACIÓN POR CLASE
=====
```

	precision	recall	f1-score	support
caminar_espalda	0.8286	0.9587	0.8889	726
caminar_frente	0.8949	0.8878	0.8913	998
girar	0.8594	0.7938	0.8253	970
ponerse_de_pie	0.5783	0.3345	0.4238	574
sentado	0.7157	0.9115	0.8018	768
accuracy			0.8038	4036
macro avg	0.7754	0.7772	0.7662	4036
weighted avg	0.7953	0.8038	0.7915	4036

Tabla resumen por clase:

Clase	Precision	Recall	F1-Score	Support
caminar_espalda	0.8286	0.9587	0.8889	726
caminar_frente	0.8949	0.8878	0.8913	998
girar	0.8594	0.7938	0.8253	970
ponerse_de_pie	0.5783	0.3345	0.4238	574
sentado	0.7157	0.9115	0.8018	768

Mejor clase: caminar_frente (F1=0.8913)

Peor clase: ponerse_de_pie (F1=0.4238)

```
print("\n" + "="*70)
print("MATRIZ DE CONFUSIÓN")
print("="*70)

# Calcular matriz
cm = confusion_matrix(y_test, y_pred)

# Visualización
fig, ax = plt.subplots(figsize=(10, 8))
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=le.classes_
```

```
)
disp.plot(cmap='Blues', ax=ax, xticks_rotation=45, values_format='d')
ax.set_title('Matriz de Confusión - Modelo Final', fontsize=14, fontweight='bold', pad=20)
ax.set_xlabel('Clase Predicha', fontsize=11)
ax.set_ylabel('Clase Real', fontsize=11)

plt.tight_layout()
output_path = f"{OUTPUT_DIR}/graficos/confusion_matrix_final.png"
plt.savefig(output_path, dpi=150, bbox_inches='tight')
plt.show()

print(f"Matriz guardada: {output_path}")

# Analizar confusiones principales
print("\nAnálisis de confusiones:")
print("-" * 70)

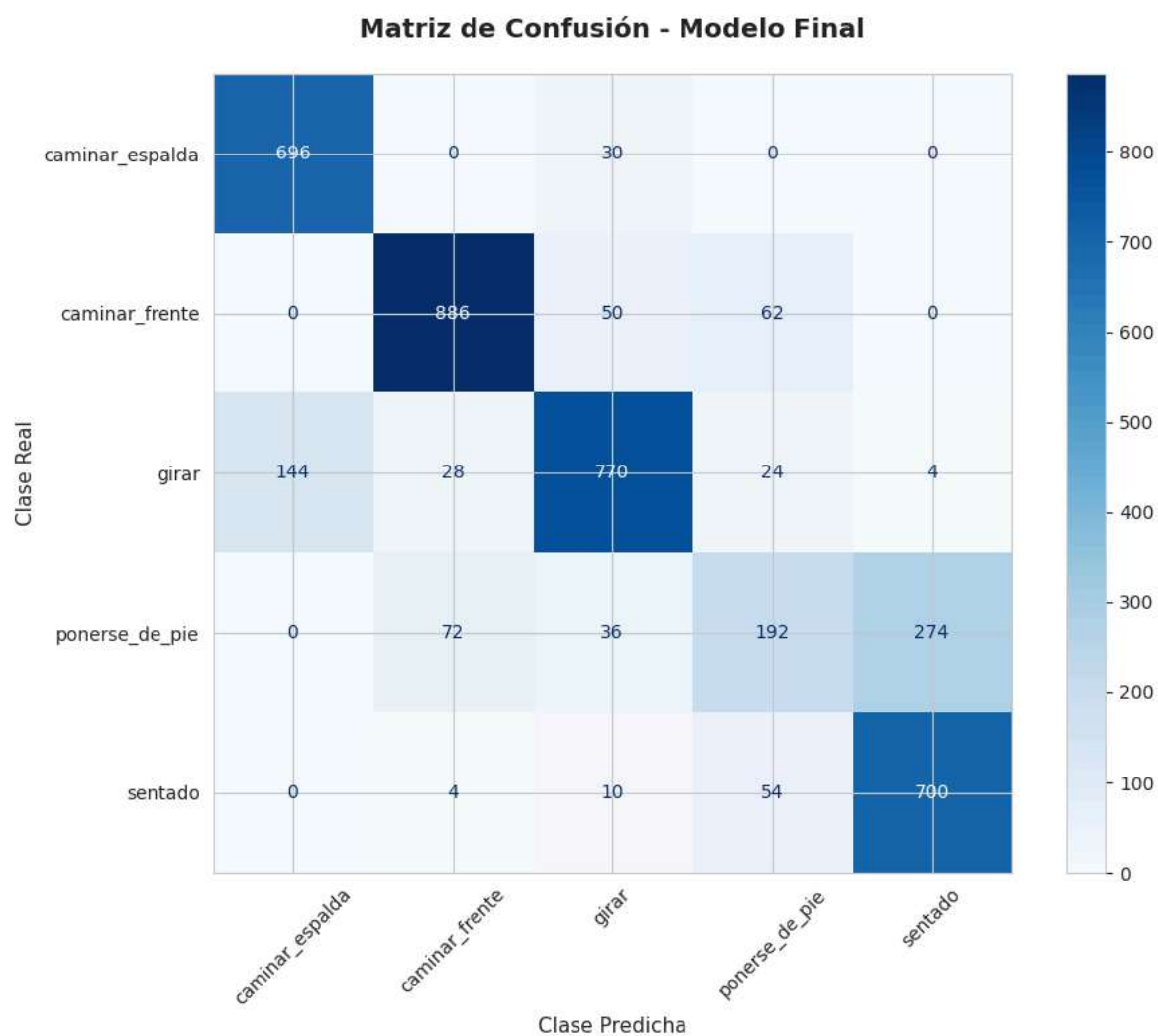
# Para cada clase, ver con qué otra se confunde más
for i, clase_real in enumerate(le.classes_):
    # Copiar fila y poner diagonal en 0 para encontrar máximo fuera de diagonal
    row = cm[i].copy()
    row[i] = 0
    max_confusion_idx = np.argmax(row)
    max_confusion_count = row[max_confusion_idx]

    if max_confusion_count > 0:
        clase_confundida = le.classes_[max_confusion_idx]
        total_clase = cm[i].sum()
        porcentaje = (max_confusion_count / total_clase) * 100
        print(f"{clase_real:<20} → confundida con {clase_confundida:<20} ({max_confusion_count}/{total_clase} = {porcentaje}%)")
```

=====

MATRIZ DE CONFUSIÓN

=====



Matriz guardada: /content/drive/MyDrive/PROYECTO APO/Entrega3/graficos/confusion_matrix_final.png

Análisis de confusiones:

```
-----
caminar_espalda    → confundida con girar          (30/726 = 4.1%)
caminar_frente     → confundida con ponerse_de_pie      (62/998 = 6.2%)
girar              → confundida con caminar_espalda     (144/970 = 14.8%)
ponerse_de_pie     → confundida con sentado             (274/574 = 47.7%)
sentado            → confundida con ponerse_de_pie      (54/768 = 7.0%)
```

```
print("\n" + "="*70)
print("ANÁLISIS DE ERRORES INDIVIDUALES")
print("="*70)

# Identificar índices de errores
error_mask = y_test != y_pred
error_indices = test_idx[error_mask]
n_errors = error_mask.sum()

print(f"Total de errores: {n_errors} / {len(y_test)} ({n_errors/len(y_test)*100:.1f}%)")

# Crear DataFrame con errores
errors_df = pd.DataFrame({
    'video': groups[error_indices],
    'frame': df.iloc[error_indices]['frame'].values,
    'clase_real': le.inverse_transform(y_test[error_mask]),
    'clase_predicha': le.inverse_transform(y_pred[error_mask]),
    'confianza': y_pred_proba[error_mask].max(axis=1)
})

print(f"\nPrimeros 20 errores:")
print(errors_df.head(20).to_string(index=False))

# Análisis por video
print(f"\nErrores por video de test:")
for video in np.unique(groups_test):
    video_errors = errors_df[errors_df['video'] == video]
```

```

video_total = np.sum(groups[test_idx] == video)
error_rate = len(video_errors) / video_total * 100
print(f" {video}: {len(video_errors)}/{video_total} errores ({error_rate:.1f}%)"

# Errores de baja confianza (modelo dudaba)
low_confidence_errors = errors_df[errors_df['confianza'] < 0.5]
print(f"\nErrores con confianza < 50%: {len(low_confidence_errors)} ({len(low_confidence_errors)/n_errors*100:.1f}% del tot

```

```

=====
ANÁLISIS DE ERRORES INDIVIDUALES
=====

```

Total de errores: 792 / 4036 (19.6%)

Primeros 20 errores:

video	frame	clase_real	clase_predicha	confianza
Video 1	43	caminar_frente	ponerse_de_pie	0.647799
Video 1	44	caminar_frente	ponerse_de_pie	0.549709
Video 1	45	caminar_frente	ponerse_de_pie	0.591098
Video 1	46	caminar_frente	ponerse_de_pie	0.625717
Video 1	47	caminar_frente	ponerse_de_pie	0.678246
Video 1	48	caminar_frente	ponerse_de_pie	0.651773
Video 1	49	caminar_frente	ponerse_de_pie	0.544240
Video 1	50	caminar_frente	ponerse_de_pie	0.471895
Video 1	51	caminar_frente	ponerse_de_pie	0.551289
Video 1	52	caminar_frente	ponerse_de_pie	0.596225
Video 1	55	caminar_frente	ponerse_de_pie	0.519841
Video 1	57	caminar_frente	ponerse_de_pie	0.417505
Video 1	150	girar	caminar_frente	0.715155
Video 1	151	girar	caminar_frente	0.595285
Video 1	152	girar	caminar_frente	0.555285
Video 1	190	caminar_espalda	girar	0.517236
Video 1	197	caminar_espalda	girar	0.595169
Video 1	258	girar	caminar_espalda	0.776590
Video 1	259	girar	caminar_espalda	0.781231
Video 1	260	girar	caminar_espalda	0.776396

Errores por video de test:

Video 1: 106/612 errores (17.3%)
Video 10: 180/656 errores (27.4%)
Video 14: 360/1262 errores (28.5%)
Video 17: 146/1506 errores (9.7%)

Errores con confianza < 50%: 190 (24.0% del total)

Comparación con entrega 2

```

print("\n" + "="*70)
print("COMPARACIÓN: ENTREGA 2 vs ENTREGA 3")
print("="*70)

comparacion = pd.DataFrame({
    'Métrica': [
        'Accuracy (test)',
        'Validación cruzada',
        'Features usadas',
        'Tiempo entrenamiento',
        'Gap train-test'
    ],
    'Entrega 2': [
        '99.8%',
        'No aplicada (train_test_split)',
        '1,629 features',
        '~2 min',
        'Desconocido (posible data leakage)'
    ],
    'Entrega 3': [
        f'{acc_test*100:.1f}%',
        f'{np.mean(cv_scores)*100:.1f}% ± {np.std(cv_scores)*100:.1f}%',
        '100 features (reducción 93.9%)',
        f'{elapsed:.1f}s',
        f'{gap*100:.1f}%'
    ]
})

print(comparacion.to_string(index=False))

print("\nInterpretación:")
print(" • Entrega 2 (99.8%): Accuracy INFLADO por data leakage")
print(" • Entrega 3 (80.4%): Accuracy REAL validado con GroupKFold")
print(" • Reducción de features: 93.9% menos columnas, mínima pérdida de accuracy")
print(" • Overfitting moderado (19.6% gap) indica que modelo memoriza algunos patrones")

```

```
=====
COMPARACIÓN: ENTREGA 2 vs ENTREGA 3
=====
```

Métrica	Entrega 2	Entrega 3
Accuracy (test)	99.8%	80.4%
Validación cruzada	No aplicada (train_test_split)	80.3% ± 3.9%
Features usadas	1,629 features	100 features (reducción 93.9%)
Tiempo entrenamiento	~2 min	26.5s
Gap train-test Desconocido (posible data leakage)		19.6%

Interpretación:

- Entrega 2 (99.8%): Accuracy INFLADO por data leakage
- Entrega 3 (80.4%): Accuracy REAL validado con GroupKFold
- Reducción de features: 93.9% menos columnas, mínima pérdida de accuracy
- Overfitting moderado (19.6% gap) indica que modelo memoriza algunos patrones

✓ Guardar Artefactos y Preparar para Despliegue

Objetivo:

Generar todos los archivos necesarios para:

- Reporte : Gráficos, tablas con métricas
- Sistema en tiempo real: Modelo entrenado, configuraciones
- Documentación: Metadata del experimento

**** Guardar modelo y componentes ****

```
print("\n" + "="*70)
print("GUARDANDO MODELO Y COMPONENTES")
print("="*70)

# Crear diccionario con todo lo necesario para inferencia
pipeline_completo = {
    'modelo': rf_final,
    'imputer': imputer,
    'top_indices': top_indices,
    'top_features': top_features,
    'label_encoder': le,
    'scaler': None, # No usamos scaler, pero lo dejamos para compatibilidad
    'metadata': {
        'fecha_entrenamiento': pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S'),
        'n_features_original': X.shape[1],
        'n_features_seleccionadas': len(top_indices),
        'clases': le.classes_.tolist(),
        'train_accuracy': train_score,
        'test_accuracy': acc_test,
        'cv_accuracy_mean': np.mean(cv_scores),
        'cv_accuracy_std': np.std(cv_scores),
        'n_samples_train': len(X_train),
        'n_samples_test': len(X_test),
        'videos_train': np.unique(groups_train).tolist(),
        'videos_test': np.unique(groups_test).tolist()
    }
}

# Guardar modelo
model_path = f"{OUTPUT_DIR}/modelo_final.pkl"
joblib.dump(pipeline_completo, model_path)
print(f"✓ Modelo guardado: {model_path}")
print(f"  Tamaño del archivo: {os.path.getsize(model_path) / 1024 / 1024:.2f} MB")
```

```
=====
GUARDANDO MODELO Y COMPONENTES
=====
✓ Modelo guardado: /content/drive/MyDrive/PROYECTO APO/Entrega3/modelo_final.pkl
  Tamaño del archivo: 14.32 MB
```

**** Guardar métricas en CSV ****

```
print("\n" + "="*70)
print("GUARDANDO MÉTRICAS EN CSV")
print("="*70)

# Métricas globales
metricas_globales = pd.DataFrame({
    'metrica': ['accuracy_test', 'f1_weighted', 'f1_macro',
```

```

        'cv_mean', 'cv_std', 'train_accuracy', 'gap_train_test',
        'n_features', 'tiempo_entrenamiento_seg'],
    'valor': [acc_test, f1_weighted, f1_macro,
              np.mean(cv_scores), np.std(cv_scores), train_score, gap,
              len(top_indices), elapsed]
})

csv_path = f"{OUTPUT_DIR}/metricas_globales.csv"
metricas_globales.to_csv(csv_path, index=False)
print(f"✓ Métricas globales: {csv_path}")

# Métricas por clase
metricas_por_clase = pd.DataFrame({
    'clase': le.classes_,
    'precision': precision,
    'recall': recall,
    'f1_score': f1,
    'support': support
})

csv_path = f"{OUTPUT_DIR}/metricas_por_clase.csv"
metricas_por_clase.to_csv(csv_path, index=False)
print(f"✓ Métricas por clase: {csv_path}")

# Resultados de validación cruzada
cv_results = pd.DataFrame(fold_info)
csv_path = f"{OUTPUT_DIR}/cross_validation_results.csv"
cv_results.to_csv(csv_path, index=False)
print(f"✓ Resultados CV: {csv_path}")

# Errores individuales
csv_path = f"{OUTPUT_DIR}/errores_detallados.csv"
errors_df.to_csv(csv_path, index=False)

```

```

=====
GUARDANDO MÉTRICAS EN CSV
=====
✓ Métricas globales: /content/drive/MyDrive/PROYECTO APO/Entrega3/metricas_globales.csv
✓ Métricas por clase: /content/drive/MyDrive/PROYECTO APO/Entrega3/metricas_por_clase.csv
✓ Resultados CV: /content/drive/MyDrive/PROYECTO APO/Entrega3/cross_validation_results.csv
✓ Errores detallados: /content/drive/MyDrive/PROYECTO APO/Entrega3/errores_detallados.csv

```

```

print("\n" + "="*70)
print("RESUMEN FINAL DEL EXPERIMENTO")
print("="*70)

print(f"""
CONCLUSIÓN.

```

No se ha podido establecer conexión con el servicio reCAPTCHA. Comprueba tu conexión a Internet y vuelve a cargar la página para ver otro reCAPTCHA.